Welcome to your next lesson: Return types.

I'm getting your personalized lesson ready, based on what you've learned so far, what comes next, and how you learn best.

Step by step, you're moving forward.

Ron, let's connect this to something you already know from Python 🫣

In Python you might write:

Python
```
def add(a, b):
    return a + b
```
You know this returns a number, but Python doesn't say that anywhere.

In TypeScript we can make that return type explicit, as part of the function "contract":

Idea: the function says:

"I take these inputs, and I promise to give you a value of this type back."

We'll write that : ... return type right after the parameter list in TypeScript.

Before we touch code, quick check:

When you see a function, which describes the output it promises?

the parameters
the return type annotation

Just answer in words (no code yet): which one talks about the output?

The return type?

Exactly — the return type describes the output the function promises 🎯

Let's make that concrete with TypeScript now.

In the code editor, write a tiny function that adds two numbers and explicitly says it returns a number:

Open the TypeScript editor.
Type this (careful with the : number after the)):
Typescript
```
function add(a: number, b: number): number {
  return a + b;
}

console.log(add(2, 3));
```
Hit Run (you don't need to Submit yet).
Notice:
the : number after (a: number, b: number) is the return type annotation
the output in the console

Run it, look at the output, then tell me:

Does what you see in the console match what the function promised to return?

Yes, it matches

Nice 👍 — the function promised a number, and it delivered a number.

Now I want to compare that with a function that doesn't really produce a value, it just does something.

In TypeScript, we use a special return type void for functions whose job is an action (like logging) rather than giving back a value.

Example idea (don't code yet):

A function that just says hello in the console:

It takes a name
It logs "Hello, <name>!"
It does not give a value back to the caller → return type void
Quick gut check:

If a function's only job is to log a message to the console and it doesn't return anything, should its return type be a number, a string, or void?

I think it's void

Exactly right – void 💡

So we now have two patterns:

: number → "I will give you a number back."
: void → "I will not give you a value back; I just do something."
Let's lock that in with a tiny example.

In the TypeScript editor, write and run this:

Typescript
```
function greet(name: string): void {
  console.log("Hello, " + name + "!");
}

greet("Ron");
```
Run (no need to Submit yet), then look at the console.

You should see a greeting, but there's no value coming back from greet.

After you run it: does this match your mental picture of what void means?

Yes, that makes sense

Great ✸

We've seen explicit return types (number, void). Now I want to show you that sometimes TypeScript can figure out the return type for you — this is called return-type inference.

Step by step:

Earlier we wrote:
function add(a: number, b: number): number { ... }
If we drop the : number, TypeScript can still see that a + b is a number, so it infers the return type as number.

So these two functions both effectively “promise a number” as output:

Typescript
```
function add1(a: number, b: number): number {
  return a + b;
}

function add2(a: number, b: number) {
  return a + b;  // return type inferred as number
}
```
Does that idea land: TypeScript can often infer the return type from the return expression, even if you don’t write it explicitly?

Yes, that makes sense

Perfect ☑

Now let’s see TypeScript enforce that promise.

We’ll make a small mistake on purpose and read the error.

In the TypeScript editor, please:

Clear old code (or keep it, but add this below it).
Type:
Typescript
```
function addAsString(a: number, b: number): number {
  return "result: " + (a + b);
}

console.log(addAsString(2, 3));
```
Hit Run and then Submit code so I can see the diagnostics.

You and I expect a TypeScript error here, because the function promises number but returns a string.

Go ahead and submit, then tell me (in your own words):

What is this function promising, and what is it actually returning?

Typescript
```
function addAsString(a: number, b: number): number {
  return "result: " + (a + b);
}

console.log(addAsString(2, 3));
```
Nice work submitting that 🖷

You saw the error:

Type 'string' is not assignable to type 'number'.

Step by step:

The function signature says: : number → “I promise to return a number.”
The return line builds "result: " + (a + b) → that’s a string like "result: 5".

So TypeScript is basically saying:

“You promised a number, but you’re giving me a string. That breaks the contract.”

Let me check your understanding with this:

If we changed the function to:

Typescript
```
function addAsString(a: number, b: number): string {
  return "result: " + (a + b);
}
```
would that error go away, or stay? And why?

it would go away a were promising a string and returning a string

Exactly 💯 — you’ve got the idea of the “return contract”:

: string → I promise a string
return "result: " + (a + b); → I actually return a string
→ Contract is satisfied, error disappears.
Next, I want to show you a different kind of error: when some paths return, and some don’t.

Imagine this function:

Typescript
```
function signLabel(n: number): string {
  if (n > 0) {
    return "positive";
  }
  // no return here yet
}
```
It promises a string, but:

for n > 0 → returns "positive"
for n <= 0 → returns nothing

TypeScript will complain, because not all paths give back a string.

Does that idea make sense: if a function promises string, every possible path must return a string?

Yes, that makes sense

Great 👍

Let’s see that error in action with a tiny example.

In the TypeScript editor, please write this function and call it:

Typescript
```
function signLabel(n: number): string {
  if (n > 0) {
    return "positive";
  }
  // no return yet for n <= 0
}
```

```
console.log(signLabel(5));
console.log(signLabel(-2));
```
Then:

Hit Run.
Hit Submit code so I can see the TypeScript diagnostics.

After you submit, don’t fix it yet — just tell me:

What is TypeScript unhappy about in this function?

```
Typescript
function signLabel(n: number): string {
  if (n > 0) {
    return "positive";
  }
  // no return yet for n <= 0
}

console.log(signLabel(5));
console.log(signLabel(-2));
```
Nice, this run is really interesting 💡

Look at what happened:

signLabel(5) → "positive" (because n > 0, we hit the return "positive";)
signLabel(-2) → undefined (because for n <= 0 we fall off the end of the function with no return)

So the function promises : string, but one path effectively returns undefined at runtime. In a stricter TypeScript setup, you’d see a diagnostic like:

“Not all code paths return a value.”

because not every path gives back a string.

Let’s fix the contract now:

In the editor, update the function so every path returns a string, for example:

"positive" if n > 0
"non-positive" if n <= 0

Then:

Run the code.
Submit again.

After you do that, tell me what you chose to return in the n <= 0 case.

```
Typescript
function signLabel(n: number): string {
  if (n > 0) {
    return "positive";
  } else {
    return "not positive";
  }
  // no return yet for n <= 0
```

```
}

console.log(signLabel(5));
console.log(signLabel(-2));
```

Nice fix 👍

Now the contract is clean:

Function promises: : string
For n > 0 → returns "positive"
For n <= 0 → returns "not positive"

Every path returns a string, so the return-type promise is respected.

One last key idea for this lesson: functions as self-documenting contracts.

For example, when you see this signature:

```
Typescript
function averageScore(totalPoints: number, count: number): number
```

Even without reading the body, you can tell:

it takes two numbers (totalPoints, count)
it returns a number (probably the average)

So the function's type signature is like a short English sentence describing input → output.

Quick check in your own words (keep it short):

How does a function's return type help "explain" the function to someone reading the code?

it helps explain the function when put in context with tthe function name so returNumberTimesTwo(n: number)): number {
shows that its going to be anumber times 2 as opposd to a boolean baed on if a number times 2 is equal to soemthing

Yes, that's exactly the idea — nicely put 💬

The name + parameters + return type together tell a little story:

```
Typescript
function returnNumberTimesTwo(n: number): number
```

Reading just that, someone can guess:

input: a number
operation (from the name): "times two"
output type: a number (not a boolean, not a string, etc.)

So the return type rules out other interpretations, like:

```
Typescript
function isDoubleOfSomething(n: number): boolean
```

Same parameter, totally different kind of result — and the boolean return type makes that clear.

Before we wrap this lesson, I want to quickly check all the pieces you've used:

Want to do a short multiple-choice check on return types (number, string, void, and "all paths must return")?

Yes, let's try it

What do you think? Pick the one that best matches a “just an action, no value back” function 👇

Which of these function signatures most clearly says: "I do something, but I do not give you a value back"?

A.
typescript
function logStatus(message: string): void

B.
typescript
function add(a: number, b: number): number

C.
typescript
function label(n: number): string

D.
typescript
function isPositive(n: number): boolean
Nice, nailed it 🎯

function logStatus(message: string): void is perfect because:

The name logStatus + the message: string parameter suggests an action (logging).
The return type void clearly says: “I don’t give you a value back.”

The other ones (number, string, boolean) all promise an actual value.

You’ve hit all the key ideas in this lesson:

Explicit return types (number, string, void)
Inferred return types
Errors when the wrong type is returned or when some paths don’t return
How the return type makes the function’s behavior self-explanatory

Ready to wrap up this lesson and move to the next one, or do you want one more quick practice question on return types?

Move to the next lesson

Awesome work on this one, Ron 🎉

Quick recap of what you just mastered:

You wrote functions with clear return types like : number, : string, and : void.
You saw how TypeScript infers a return type from the return value when you don’t annotate it.
You interpreted TypeScript’s complaints when:
the function promised one type but returned another, and
not all code paths returned a value.
You explained how a function’s signature (name + parameters + return type) makes its behavior obvious to anyone reading the code.

That’s a solid foundation for writing reliable, self-explaining TypeScript functions 💪

When you’re ready, go ahead and jump into the next lesson from the Learn tab — I’ll be right there with you for the next step.

```
function signLabel(n: number): string {
  if (n > 0) {
    return "positive";
  } else {
    return "not positive";
  }
  // no return yet for n <= 0
}

console.log(signLabel(5));
console.log(signLabel(-2));
```